



JevAny: Toward General Decision Intelligence

A calibrated decision layer for models, agents, and environments

Abstract. JevAny converts pretrained language-model backbones into bounded decision models that score explicit candidate actions without autoregressive answer generation. The system supports pointer and direct-token readouts, LoRA adaptation across four released backbones, native image and video inputs on compatible checkpoints, calibrated probability outputs, and a common training and inference interface. This report specifies the model contract, architecture, objectives, production recipes, evaluation protocol, controlled ablations, compute accounting, limitations, and open research questions. The strongest release, Qwen3.8 27B LoRA, reaches **85.76% Transfer accuracy** and **90.48% JevBench accuracy**.

Artifacts: github.com/SimpleJev/JevAny · [Hugging Face model collection](#)

1 Introduction

JevAny turns a language-model backbone into a bounded decision model. It consumes a shared state, one or more questions, and explicit candidate answers; it returns a normalized distribution over the supplied candidates without decoding answer text. The current release trains rank-8 LoRA adapters in BF16 with a cross-entropy objective and either a residual pointer head or a direct-token readout. The strongest released model, Qwen3.8 27B LoRA, reaches **85.76% Transfer accuracy** and **90.48% JevBench accuracy**.

The release family spans Gemma 4B, Qwen3.5 4B, Qwen3.8 27B, and Muse Glimmer 30B. The training corpus contains **1,772,725 text records** and **2,180,242 labelled decisions** across preference, agent/tool decision, reasoning, classification, and safety tasks. This report describes the corpus scale and task families.

The ablation program used fixed-step screens, held-out calibration, paired record-group bootstrap intervals, explicit promotion gates, and full-run confirmation. Pure InfoNCE, supervised Brier regularization, RLCR variants, targeted mixture changes, and dual-tower alternatives failed their predeclared promotion gates.

2 Problem formulation

For a state s , question q , and explicit choices $\mathcal{C} = \{c_1, \dots, c_K\}$, JevAny predicts

$$p(y = i \mid s, q, \mathcal{C}), \quad i \in \{1, \dots, K\}.$$

The candidate set is part of the model input, and the output is already a probability simplex suitable for ranking, confidence thresholds, or downstream control.



2.1 Structured causal input

The text layout is

$$\langle \text{state} \rangle s \langle \text{question} \rangle q \prod_{i=1}^K (\langle \text{option} \rangle c_i \langle / \text{option} \rangle) \langle \text{decide} \rangle.$$

User text is escaped so it cannot forge control delimiters. When a backbone has validated custom-mask support, questions may be packed with a block-causal mask. Recurrent, sliding-window, and other unsupported architectures instead receive independent causal rows that repeat the shared state. This preserves question-level isolation across backbone families.

2.2 Pointer readout

Let h_d be the hidden state at the decision marker and h_i the state at the closing marker of option i . With pointer width $d_p = 256$,

$$q_d = W_q h_d + b_q, \quad k_i = W_k h_i + b_k, \quad z_i = \frac{k_i^\top q_d}{\sqrt{d_p}}.$$

The released pointer checkpoints add a zero-output-initialized residual scorer,

$$z_i \leftarrow z_i + w_2^\top \text{GELU}(W_1(k_i \odot q_d) + b_1) + b_2,$$

where the residual width is 256. Softmax is applied only across the options in the request. Consequently, the head has no fixed option-vocabulary ceiling; the practical choice limit is the model context window.

2.3 Direct-token readout

Direct-token mode prefixes candidates with a deterministic table of distinct, round-tripping single-token labels. At the decision marker, the frozen base LM head produces full-vocabulary logits v and training minimizes

$$\mathcal{L}_{\text{DT}} = - \sum_{i=1}^K t_i \log \text{softmax}(v)_{\nu_i},$$

where ν_i is the vocabulary ID assigned to option i . Inference selects the K candidate logits and renormalizes them. The released implementation supports up to 255 candidates.

Pointer vs direct-token at a glance

	Pointer	Direct-token
Scoring	Learned query-option compatibility	Frozen LM head over fixed label tokens
Training	Candidate-only softmax; efficient	Full-vocabulary normalization; slower
Choices	More than 255, context permitting	Up to 255
Inference	One backbone prefill, no decoding	One backbone prefill, no decoding
Observed result	4B Better Transfer: 78.68%	Better JevBench: 80.95%

Both paths have similar expected inference latency for comparable inputs because neither autoregressively generates an answer. In the complete Qwen 4B training trajectory, direct-token consumed 381.9 H200 GPU-hours versus 330.6 for pointer, about 15.5% more; the released direct-token checkpoint was selected earlier in that trajectory.

2.4 System contract and multimodal path

JevAny exposes the same decision contract during training, offline evaluation, and serving. A request contains shared state, one or more questions, and an ordered candidate set for each question. A response contains the selected index, the corresponding candidate value, and a normalized probability for every candidate. The implementation maintains the following invariants:

- **Bounded outputs.** The model can select only from caller-supplied candidates; it does not generate an unlisted answer.
- **Native multimodality.** Images and videos are resolved, validated, and passed through the loaded backbone’s native processor. The same decision schema is used for text-only and multimodal requests.
- **Order-aware supervision.** Labels refer to candidate indices after serialization, and the same order is retained through scoring and metrics.
- **Question isolation.** Packed execution is used only when the backbone supports the validated mask; otherwise each question receives an independent causal row.
- **Probability preservation.** Saved evaluation artifacts retain the full candidate distribution needed for NLL, Brier, ECE, and later audits.
- **Self-describing checkpoints.** Adapter configuration, special-token state, readout metadata, base revision, learned head weights, and declared media capabilities travel with the checkpoint.
- **No-decoding inference.** Both readout families require one backbone forward pass and return scores directly, avoiding answer parsing and decoding ambiguity.

The built-in serving path accepts trusted local image and video inputs, exposes active media capabilities through model introspection, and rejects unsupported media rather than silently dropping it. This design lets text, vision, and video backbones share one decision API while retaining their native preprocessing.

3 Training method

3.1 Supervised objective

For a hard label y , the default objective is

$$\mathcal{L}_{\text{CE}} = -\log p_y.$$

For a normalized soft target t , it becomes

$$\mathcal{L}_{\text{soft}} = -\sum_i t_i \log p_i.$$

The release runs use CE, rank-8 LoRA over the backbone’s supported linear projections, zero LoRA dropout, a one-cycle learning-rate schedule, BF16 weights and autocast, and seed 0. Pointer models train the 256-dimensional residual head alongside LoRA. Direct-token freezes the original LM head and trains only the adapter path.

3.2 Optional research objectives

The training code also supports contrastive terms, supervised Brier penalties, and an experimental calibration-aware policy stage. These were ablated, but are not part of the five current releases.

For RLCR, noisy logit proposals $z' = z + \epsilon$ use $\epsilon \sim \mathcal{N}(0, \sigma^2 I)$ after per-proposal mean centering. If $a = \arg \max_i \text{softmax}(z')_i$, confidence is $q = \text{softmax}(z')_a$, correctness is $c = t_a$, and reward is

$$r(c, q) = c - (q - c)^2.$$

A group-relative advantage multiplies the Gaussian location score, while a CE anchor limits decision drift. Policy optimization operates directly on perturbed decision logits, without generating reasoning rollouts.

3.3 Production release recipe

Model	Readout	LoRA	Global batch	GPUs	Released step
Gemma 4B	Pointer	8	384	32	2,771
Qwen3.5 4B	Pointer	8	128	32	13,850
Qwen3.5 4B	Direct-token	8	128	32	9,695
Muse Glimmer 30B	Pointer	8	160	40	3,324
Qwen3.8 27B	Pointer	8	32	32	22,160

All five runs use one epoch, learning rate 5×10^{-5} , weight decay 0.01, BF16 training, gradient checkpointing, and LoRA targets resolved from the loaded architecture. The release data scale is fixed at 1,772,725 text records and 2,180,242 labelled decisions.

4 Evaluation protocol

4.1 Suites

- **Transfer** contains 1,046 clean, knowable decisions spanning seven cross-domain datasets and four robustness slices. Every reported run covers all 1,046 scored items.
- **JevBench** contains all 231 items in the frozen public development conversion, yielding public-development scores.

Transfer accuracy is an item-level mean. NLL, multiclass Brier score, and 10-bin ECE use the candidate probabilities. A scalar temperature is fit on a separate calibration partition by micro NLL and then applied unchanged to the evaluation rows. Temperature scaling cannot change argmax accuracy.

For controlled ablations, comparisons use identical rows and option order. Uncertainty is estimated by paired, source-stratified record-group bootstrap; sibling questions remain together. Registered studies use 2,000 or 5,000 resamples and apply explicit non-inferiority and calibration-improvement gates.

4.2 Released model results

Model	Transfer $\uparrow$	JevBench $\uparrow$	NLL $\downarrow$	Brier $\downarrow$	ECE $\downarrow$
Gemma 4B LoRA	70.84	77.49	0.706	0.369	0.056
Qwen3.5 4B LoRA	78.68	80.09	0.587	0.297	0.035
Qwen3.5 4B Direct-Token	78.20	80.95	0.564	0.291	0.029
Muse Glimmer 30B LoRA	83.46	87.45	0.464	0.229	0.032
Qwen3.8 27B LoRA	85.76	90.48	0.392	0.200	0.030

All accuracy values are percentages. NLL, Brier, and ECE are Transfer metrics. Unrounded values and suite hashes live in [the machine-readable release record](#).

5 Ablation program

5.1 Ablation procedure

The ablation program screened candidates at fixed budgets and confirmed them at full scale:

1. **Register** the arm, frozen inputs, causal comparison step, metrics, and promotion gate before training.
2. **Audit step zero** for identical predictions or initialized weights whenever an arm reuses a parent.
3. **Screen at fixed budget** so capacity, head, readout, and objective candidates receive matched exposure.
4. **Recalibrate independently** on held-out calibration rows; never fit temperature on Transfer or JevBench.
5. **Compare paired rows** with group bootstrap intervals and reject an arm if either accuracy non-inferiority or calibration improvement fails.
6. **Confirm at scale** before promoting a screen winner to the release recipe.

The early 4B sweep favored rank 32, query-mean, and a small contrastive weight by aggregate rank at its fixed causal checkpoint. The separate full model-family runs use rank 8, decision-marker/option-close, and CE, which is the configuration recorded by the published checkpoints.

5.2 Run validity and metric corrections

One CE-path isolation run was marked invalid after its parity audit and is excluded from all conclusions. Capacity probes, failed launches, and telemetry-recovery jobs likewise do not enter the result tables. A post-terminal audit also found that two early reviews had reported raw rather than temperature-scaled NLL because saved logits took precedence over calibrated probabilities in the metric helper. The amended NLL values are used here; accuracy, Brier, ECE, promotion gates, and selected arms were verified unchanged.

5.3 Pointer-head structure

A controlled 250-step screen compared the dot-product head, an MLP scorer, and a zero-initialized residual correction. The table shows calibrated development metrics over 904 decisions.

Head	Accuracy ↑	NLL ↓	Brier ↓	ECE ↓	Parameters
Linear pointer	81.97	0.4776	0.2603	0.0671	2,621,952
MLP scorer	82.41	0.4935	0.2596	0.0722	2,688,001
Residual pointer	82.63	0.4586	0.2497	0.0601	2,688,001

The residual arm passed the registered gate and was selected. Its final layer starts at zero, preserving the base pointer score at initialization while allowing a nonlinear correction during training.

5.4 Representation readout

The early Qwen 4B study compared the decision marker and option closing marker against mean-pooled query, option, and combined representations. Results below are the fixed-screen development values.

Readout	Accuracy ↑	NLL ↓	Brier ↓	ECE ↓
Decision / option-close control	77.88	0.5405	0.3065	0.0642
Query-mean	78.87	0.5117	0.2991	0.0428
Option-mean	74.45	0.6422	0.3653	0.0514
Combined means	73.34	0.6322	0.3709	0.0628

Query-mean led this early screen. Full family training later retained decision-marker / option-close as the stable cross-backbone recipe.

5.5 LoRA rank

The two studies below use different bases, budgets, and selection rules.

Qwen 4B early screen. At the fixed causal checkpoint, aggregate ranking over development and diagnostic Transfer metrics selected rank 32.

Rank	Development		Transfer diagnostic	
	Accuracy	NLL	Accuracy	NLL
2	76.11	0.6385	67.59	0.9183
4	76.33	0.6187	68.55	0.9013
8	77.88	0.5405	70.08	0.8642
16	78.87	0.5128	69.69	0.8601
32	77.99	0.5010	70.27	0.7800

27B base-start gate. A separate 250-step, single-seed study compared ranks 8, 16, 32, and 64. Rank 16 was the registered control and the only eligible arm after multiplicity-corrected gates; rank 8 had better point estimates but did not pass the adjusted promotion rule.

Rank	Dev. accuracy ↑	Dev. NLL ↓	Dev. Brier ↓	Gate
8	83.63	0.4445	0.2467	not eligible
16	82.08	0.4708	0.2568	selected control
32	82.08	0.4788	0.2599	not eligible
64	82.52	0.4805	0.2579	not eligible

The production family ultimately uses rank 8 based on its separate full-run cost/quality trade-off.

5.6 Loss and calibration objectives

The objective studies covered CE, pure InfoNCE, mixed CE-InfoNCE, supervised Brier penalties, and RLCR-style policy terms.

- In the controlled 27B base-start comparison, CE and pure InfoNCE had the same corrected development NLL to four decimals (0.4766), but InfoNCE worsened Brier from 0.2592 to 0.2700. CE was selected.
- In the early 4B sweep, contrastive weights from 0.01 through 3.0 were tested. A weight of 0.03 ranked first on the development aggregate, but its diagnostic Transfer accuracy was 67.97%, below CE's 69.41%. The production release therefore uses CE.
- A rank-16 factorial screen selected residual + CE + 0.1 contrastive weight within that local screen. Effects were small: under the residual head the contrastive term changed accuracy by -0.11 points and Brier by -0.00157 . This did not override the later full-run recipe.
- Supervised Brier weights 0.1, 0.25, and 0.5 produced no arm satisfying the registered joint accuracy/calibration gate.
- Correctness-only, selected-action Brier, full-distribution Brier, and a calibration-penalized RLCR variant were separated in staged controls. No RLCR arm passed the complete promotion gate. In the earlier released SFT/RLCR pair, RLCR changed Transfer accuracy by -0.10 points (95% paired interval $[-0.58, +0.39]$) while the small development-NLL gain did not survive independent recalibration.

5.7 Data-mixture interventions

The data studies changed broad task balance while holding the model and training budget fixed. They are reported only at the aggregate level here.

- Two targeted-text variants were tested against the same control at the fixed checkpoint. Variant v5 failed both calibrated-Brier improvement and accuracy non-inferiority; variant v6a improved its accuracy point estimate by about one point but still failed the calibrated-Brier promotion gate.
- A matched whole-mixture intervention and a matched hard-reasoning intervention were compared with their frozen control. Neither passed all registered gates, so no candidate was selected.
- A proposed dose study was stopped before training because no eligible fresh selection panel was available. Its status is a stopped protocol.

5.8 Alternative decision architectures

An exploratory route compared a jointly encoded cross scorer with dual-tower alternatives on the same 1,046 Transfer decisions.

Architecture	Accuracy ↑	NLL ↓	Brier ↓	ECE ↓
Cross-joint	57.17	1.1766	0.5915	0.1494
Scalar dual tower	35.28	1.2459	0.6633	0.0136
Expressive dual, scratch	47.42	1.1958	0.6176	0.0717
Expressive dual, official init	41.78	1.2351	0.6390	0.0944

The cross-joint package beat the original dual tower, and the expressive scratch dual improved substantially over the scalar dual. Nevertheless, the complete registered gates failed: one dual comparator did not establish learning, and the rescued variants did not jointly pass learning and cross-comparison conditions. No architecture was promoted. These single-seed comparisons used packages that differ from the released LoRA system.

A separate LM-token package compared the official next-token objective, InfoNCE-soft, and hard/soft SoftCE variants. SoftCE-soft was promoted within that local screen (43.69% accuracy, 1.2627 NLL, 0.6501 Brier after calibration), but the preregistration labels its comparison with JevAny as a same-benchmark system comparison.

5.9 Readout family and backbone scale

The final release provides the cleanest practical readout comparison because the two Qwen 4B models share the same base revision, data scale, rank, optimizer family, and precision. Direct-token gains 0.86 JevBench points and improves all three Transfer calibration metrics, while pointer gains 0.48 Transfer accuracy points, supports more choices, and trains faster over a full trajectory.

Across released pointer models, scaling/backbone choice matters substantially: Gemma 4B reaches 70.84% Transfer, Qwen 4B 78.68%, Muse 30B 83.46%, and Qwen 27B 85.76%. This family comparison varies architecture and parameter count together.

5.10 Ablation ledger

Track	Arms	Outcome
Pointer head	Linear, MLP, residual	Residual selected in the controlled screen.
Representation	Marker/close, query mean, option mean, combined	Query mean led the early screen; marker/close retained for the full family.
LoRA rank	2/4/8/16/32 at 4B; 8/16/32/64 at 27B	Local winners depended on protocol; production uses rank 8.
Contrastive loss	CE, InfoNCE, seven mixed weights	Small mixed terms helped some calibration screens; CE retained at full scale.
Supervised Brier	Weights 0.1/0.25/0.5 vs CE	No weight passed the joint gate.
Post-training	CE-path controls, correctness-only, selected/full Brier RLCR	No policy arm promoted; RLCR remains experimental.
Data mixture	Control, targeted variants, matched package/hardness changes	No candidate passed all gates; one planned dose study stopped pre-run.
Architecture	Cross-joint, scalar dual, expressive dual, initialized dual	Informative package differences, but complete promotion gates failed.
LM-token objective	Official CLM, InfoNCE-soft, SoftCE hard/soft	SoftCE-soft won its local screen; treated as a system-package result.
Decision readout	Pointer vs direct-token	Complementary 4B trade-off; both released.
Backbone family	Gemma, Qwen, Muse	Qwen3.8 27B is the highest-accuracy release.

6 Compute accounting

Compute below covers training through each released checkpoint. GPU-hours equal elapsed wall-clock hours times the actual DDP world size and exclude ablation, evaluation, failed jobs, and training after the selected checkpoint.

Model	Step	Parallel H200s	Wall hours	H200 GPU-hours
Gemma 4B LoRA	2,771	32	~3.28	~104.9
Qwen3.5 4B LoRA	13,850	32	10.33	330.6
Qwen3.5 4B Direct-Token	9,695	32	8.41	269.1
Muse Glimmer 30B LoRA	3,324	40	2.88	115.4
Qwen3.8 27B LoRA	22,160	32	18.83	602.7
Released-checkpoint total				~1,423

Gemma is approximate because that earlier checkpoint format did not persist cumulative elapsed seconds; its value uses the recorded run start and checkpoint timestamp. Direct-token, Qwen 27B, and Muse use checkpoint-native elapsed time; Qwen 4B pointer uses terminal trainer telemetry because the released checkpoint is the completed step 13,850 run. Peak within-run parallelism was 40 GPUs.

7 Reproducibility and evaluation scope

Release checks. Base revisions are pinned. Every public artifact carries its adapter metadata and tokenizer state. Release manifests, checkpoint-native reload reports, and GPU loader/reconstruction parity were checked before publication. Transfer covers 1,046/1,046 scored decisions and JevBench covers 231/231 public development items.

- Short screens use a single training seed unless stated otherwise. Measuring training-seed uncertainty requires repeated runs.
- Transfer served as a development diagnostic in several research stages and is reported on that basis in the release comparison.
- Backbone families differ in weights and may differ in architecture, tokenizer, and native media machinery.
- GPU-hours cover training through each selected checkpoint. Total research spend and inference cost are outside this accounting.
- The current release contains LoRA SFT checkpoints. Full-parameter SFT, improved post-training, and hyperparameter-optimized variants are roadmap items.

8 Open research questions

The broader opportunity is to make decision models a reusable systems layer between foundation models, agents, tools, and environments. Generative models can interpret context and propose possibilities; a Jev-style model can compare explicit alternatives, quantify uncertainty, enforce an action boundary, and decide when to defer. A compact Jev paired with a larger LLM—for example, a 4B decider with a 27B reasoner—is one instance of this broader architecture.



- **General decision models.** Can one decision layer transfer across domains, modalities, action spaces, and backbone families? What scaling laws and choice-set invariants should govern such models?
- **Reasoning and agent bridges.** How should Jev rank proposals, arbitrate specialists, veto unsafe actions, or route uncertain cases to more capable reasoning without double-counting correlated evidence?
- **Open-world and long-horizon control.** How can a bounded decider detect missing actions, return to exploration, and assign delayed credit over plans, tool calls, and environment transitions?
- **Multimodal and embodied decisions.** How should text, images, video, spatial state, and sensor streams be fused and reused across Jev and upstream agents while retaining calibrated action probabilities?
- **Learning, calibration, and safety.** How can distillation, preference learning, online feedback, abstention, and human escalation improve the system under distribution shift without feedback loops or reward hacking?
- **Systems and evaluation.** What architecture best balances quality, latency, memory, and cost? Evaluation must move beyond static accuracy to closed-loop success, action coverage, calibration, robustness, unsafe-action avoidance, and escalation quality.

8.1 Release roadmap

Planned work covers heterogeneous Jev-LLM/agent bridges, full-parameter SFT, and improved post-training and hyperparameter-optimized variants. New artifacts will be released after recipe, reconstruction, accuracy, and calibration checks.

JevAny Technical Report -- Agent-Harness Appendix

SimpleJev

Technical report appendix

Evidence snapshot: 30 September 2026 Appendix revision: 1 October 2026 Scope: bounded Jev decision-making inside LLM agents

This appendix extends the released nine-page **JevAny Technical Report**. The original PDF is preserved unchanged. It studies a specific systems question: can an LLM retain task reasoning and completion ownership while delegating bounded, basic decisions to Jev?

A. Executive conclusion

Jev can benefit an LLM agent when the LLM already has a viable task strategy and the remaining work contains repeated, low-risk choices with several real alternatives. In that regime, Jev is a bounded selection layer: the LLM interprets the task, performs difficult edits, diagnoses failures, verifies the result, and decides when the task is complete. Candidate source depends on the protocol: environments or the DOM supply controlled/web actions, while the LLM generates terminal menus. Jev only chooses within the bounded menu it receives.

The benefit is conditional rather than universal. The strongest repeated result is GPT-5.6-sol on FrozenLake: success stayed at 100% while frontier calls, tokens, and wall time fell by 64.4%, 63.1%, and 37.6%. By contrast, Sokoban usually became more expensive, the fixed six-task WebArena aggregate preserved success but used more calls and tokens, and both medium Terminal-Bench tasks failed with and without Jev. Jev cannot repair a missing global plan or add capability that is absent from the frontier model.

The practical objective is therefore not maximum delegation. It is maximum replacement of expensive frontier decisions subject to preserved verifier quality, bounded risk, and positive end-to-end efficiency.

B. System boundary and comparison

The LLM-generated terminal harness exposes the following loop:

1. The frontier LLM reads the task and current observation.
2. It emits a public reasoning summary, subgoal, one to four candidate actions, expected outcomes, and a `frontier_choice` fallback.
3. The frontier is instructed to mark precise edits, irreversible or security-sensitive operations, exception recovery, and completion as non-routine; singleton actions are locally excluded from delegation.
4. For an eligible routine decision, Jev scores only the supplied candidates.
5. The harness checks the returned index and confidence, executes the selected action, and returns the observation to the LLM.
6. The LLM interprets the observation, revises the plan, verifies the artifact, and calls completion. An independent environment verifier determines reward.

Routine eligibility is model-declared, not a semantic safety proof. The local harness enforces schema, menu cardinality, confidence fallback, a short sequence cap, and narrow command checks; it cannot generally determine whether arbitrary shell commands are reversible or safe. Terminal-Bench runs inside an isolated task container, but this protocol is not a general-purpose command sandbox.

Its D0 baseline is LLM-only under the same candidate protocol: the LLM generates the candidates and the harness executes its `frontier_choice`. Within this terminal protocol, the intervention changes who selects eligible actions, not who reasons about the task. It is not an unconstrained text-only shell baseline, and separate runs can still produce different stochastic trajectories.

The repeated cross-domain D2 matrix uses a different integration. FrozenLake, Sokoban, WebShop, and WebArena expose candidates from the environment or DOM. The optional agent adds a delegation tool and its system instructions; the baseline acts directly without that tool. Those paired rows therefore measure the complete optional-harness intervention, including its prompt/tool surface, not a selector-only ablation. Results from the two protocols are reported separately. D2 uses benchmark-sandbox action menus without a semantic risk filter and may delegate terminal UI controls such as `buy now`; it is not a production authorization or safety boundary.

C. Two orthogonal levels

Task openness and Jev autonomy are independent. A hard task can use no delegation, while a controlled task can delegate nearly every action. Results must be indexed by both axes rather than collapsed into one "easy-to-hard" scale.

- Task tier: T0 controlled; Environment property: Small deterministic state/action space; Examples: FrozenLake, Sokoban; Primary stressor: Selection mechanism and paired-seed regressions
- Task tier: T1 bounded simulator; Environment property: Longer plans but finite affordances and re-settable state; Examples: ALFWorld; Primary stressor: Subgoal order and state tracking
- Task tier: T2 structured interactive; Environment property: Open-text query with bounded interface controls; Examples: WebShop; Primary stressor: Search and refinement
- Task tier: T3 dynamic web; Environment property: Heterogeneous live UI state, popups, accounts, navigation; Examples: WebArena fixed sample6; Primary stressor: Robust browser interaction
- Task tier: T4 open terminal; Environment property: Open commands, persistent side effects, diagnosis, editing; Examples: Terminal-Bench 2; Primary stressor: General software/system completion

ALFWorld belongs at T1, not at the top of the ladder: it can require long plans and object-state tracking, but its action grammar and simulator are bounded. T4 is further stratified by a frozen historical frontier pass rate: easy (4/4), medium (2/4), and hard (0/4). These are sampling strata, not post-hoc claims about Jev. ALFWorld has not been run in the current formal matrix, so this is a protocol placement rather than an empirical Jev result.

- Delegation level: D0 direct baseline; Jev role: None; Execution: Execute LLM frontier_choice; Current evidence: Implemented
- Delegation level: D1 shadow judge; Jev role: Score but do not execute; Execution: Log agreement/regret; Current evidence: Proposed
- Delegation level: D2 selective; Jev role: LLM opts in; Execution: One bounded action with fallback; Current evidence: 96-pair matrix
- Delegation level: D3 routine default; Jev role: All eligible basic decisions; Execution: Normally one action; at most two if composable; Current evidence: Terminal-Bench harness
- Delegation level: D4 bounded subgoal; Jev role: Repeated selection from an LLM-authored library; Execution: Stop on novelty, risk, or verification; Current evidence: Proposed

Configured delegation and realized delegation are different quantities. Report both eligible coverage and all-command replacement. Neither is automatically a frontier-call saving: D2 or D3 can still call the LLM before every choice. D4 is the intended design for amortizing one expensive planning call over several safe local decisions, but it has not yet been evaluated.

Decision significance is a third reporting axis:

- Level: A0 cosmetic; Meaning: Paraphrases or operationally equivalent commands; Default policy: Do not delegate; collapse the menu
- Level: A1 diagnostic; Meaning: Different information sources or verification methods; Default policy: Primary Jev target
- Level: A2 path branch; Meaning: Different products, routes, or recoverable local approaches; Default policy: Delegate only with immediate feedback and fallback
- Level: A3 strategic/high risk; Meaning: Architecture, exact edits, credentials, destructive or irreversible actions; Default policy: Keep with the frontier LLM

Most current open-agent evidence is A1, with limited A2 navigation. A high candidate count alone is therefore not evidence of high-level agent autonomy.

D. Practical operating principles

D.1 Delegate only eligible decisions

Delegate when all of the following are true:

- the choice is routine, locally decidable, low risk, and reversible;

- the menu contains at least two valid alternatives with materially different observations or local paths;
- the current observation contains enough information to rank them;
- a wrong choice can be detected and recovered without corrupting the task;
- the cost of Jev selection is lower than another frontier reasoning call.

Inspection, navigation, waiting versus polling, compilation, and choosing a verification method are good candidates. Exact file contents, architectural strategy, destructive commands, security-sensitive actions, and final completion remain with the frontier LLM.

D.2 Design choices, not paraphrases

Candidate sets should be small (normally two to four), executable, and outcome-oriented. "Inspect raw bytes," "query the database schema," and "check available tools" are useful alternatives because they expose different evidence. Three paraphrases of ls are not. Each menu must preserve a safe LLM fallback and should state the expected observation. A routine singleton is not a decision; the final strict protocol rejects it and asks the LLM to regenerate a real menu.

D.3 Batch only state-compatible actions

Batch decisions only when the LLM explicitly marks them composable and an earlier action cannot invalidate the later candidate. The current terminal protocol caps a delegated sequence at two commands and stops on no progress, an incomplete command, novelty, or changed state. Early unlimited sequences were harmful: on the first regex-log prototype, both modes passed, but optional delegation increased frontier calls from 20 to 30 and tokens from 231,070 to 395,501.

D.4 Make fallback a first-class action

Jev may select only a valid candidate index. On malformed output, service failure, or confidence below 0.55, the harness executes the LLM's fallback. This bounds selection risk but does not remove the latency of a failed Jev request. The SQLite strict smoke passed with one low-confidence fallback; four of five eligible menus were actually delegated.

D.5 Respect the capability ceiling

Jev compares options; it does not invent the missing option or recover a task strategy that the LLM did not express. If the frontier cannot formulate the configuration plan, database recovery algorithm, or correct code edit, better local selection is insufficient. This was visible in both medium Terminal-Bench tasks: 74–79% command replacement did not change 0% success and made the runs more expensive.

D.6 Optimize the frontier, not delegation rate

More delegation is useful only if verifier quality survives and total calls, tokens, cost, or time improve. A point is dominated when another point has no worse verifier quality and no greater resource use. Delegation coverage is an annotation, not an objective. Select demonstrations from a frozen sample and show every negative row that informed the choice.

D.7 Measure quality and efficiency together

Every result should include task and delegation tier, protocol version, exact frontier/Jev checkpoints, verifier reward, termination reason, frontier calls, tokens, estimated API cost, wall time, Jev calls/latency, eligible coverage, command replacement, repairs, failures, and fallbacks. Positive "saved" values mean 1 – optional / baseline. Jev GPU serving cost is not monetized in the current results and must remain a separate axis. A one-attempt result is a mechanism demonstration, never a population estimate.

E. When Jev helps and when it does not

- Dimension: Frontier capability; Conditions associated with benefit: LLM has a sound global plan and can interpret observations; Conditions associated with failure or overhead: Missing plan,

wrong edit, or insufficient domain capability; Evidence: SQLite succeeds; both medium TB2 tasks fail

- Dimension: Choice structure; Conditions associated with benefit: Two to four real, locally rankable alternatives; Conditions associated with failure or overhead: Singleton, paraphrases, or an omitted correct action; Evidence: SQLite raw/schema/tools menu; regex v4 had zero realized coverage
- Dimension: Risk and feedback; Conditions associated with benefit: Reversible action with immediate observation; Conditions associated with failure or overhead: Irreversible action or delayed/ambiguous feedback; Evidence: Inspection/navigation work better than precise edits
- Dimension: Repetition; Conditions associated with benefit: Many similar basic decisions, or a safely composable short sequence; Conditions associated with failure or overhead: Every action needs fresh global reasoning; Evidence: FrozenLake wins; WebArena aggregate rarely delegates
- Dimension: Cost balance; Conditions associated with benefit: Frontier call is expensive relative to Jev; Conditions associated with failure or overhead: Task action is already trivial and short; Evidence: FrozenLake benefits; Sokoban token use rises
- Dimension: Environment stability; Conditions associated with benefit: Candidate meaning survives until execution; Conditions associated with failure or overhead: Popups, changing pages, or stateful side effects invalidate candidates; Evidence: Popup-aware smoke works; aggregate WebArena regresses
- Dimension: Evaluation; Conditions associated with benefit: Independent verifier and paired repetitions; Conditions associated with failure or overhead: Self-reported completion or one stochastic trajectory; Evidence: 96-pair matrix versus exploratory TB2 rows

The observed task pattern is correspondingly mixed:

- Successful efficiency use: FrozenLake with GPT-5.6-sol and 27B Jev; bounded direction choices preserve success while replacing costly calls.
- Promising but not confirmed: WebShop moves from 50% to 60% success with small resource savings, but the paired intervals cross zero.
- Useful terminal mechanism: SQLite recovery preserves reward across a three-point delegation-rate sweep while the sampled 100% target uses fewer frontier resources. It remains a single-attempt curve.
- Quality-recovery signal: query-optimize changes from failure to success with slightly fewer tokens and cost, but wall time increases. One run cannot establish causality.
- Overhead-dominated: Sokoban is too cheap and simple to justify the added choice-generation/Jev path.
- Capability-limited: configure-git-webserver and llm-inference-batching-scheduler fail in both conditions despite high Jev command share.
- No aggregate web gain: the frozen six-task WebArena row keeps success at 50% but increases calls, tokens, and estimated cost.

F. Repeated cross-domain matrix

The frozen D2 matrix contains 96 paired comparisons (192 agent episodes) across ten frontier/benchmark/Jev cells. FrozenLake and Sokoban are controlled mechanism tests; WebShop and WebArena are interactive-agent checks. The WebArena row is the predeclared tasks [264, 278, 300, 158, 4, 763], not the full benchmark.

- Frontier: Opus 4.7; Task: FrozenLake; Jev: 4B; Pairs: 10; Success D0→D2: 100%→90%; Calls saved: +55.9%; Tokens saved: +51.4%; Time saved: -14.0%
- Frontier: Opus 4.7; Task: FrozenLake; Jev: 27B; Pairs: 10; Success D0→D2: 100%→90%; Calls saved: +51.5%; Tokens saved: +44.6%; Time saved: -21.8%

- Frontier: Opus 4.7; Task: Sokoban; Jev: 4B; Pairs: 10; Success D0→D2: 100%→100%; Calls saved: -2.1%; Tokens saved: -33.2%; Time saved: -28.7%
- Frontier: Opus 4.7; Task: Sokoban; Jev: 27B; Pairs: 10; Success D0→D2: 100%→100%; Calls saved: -29.5%; Tokens saved: -84.4%; Time saved: -51.0%
- Frontier: Opus 4.7; Task: WebArena; Jev: 27B; Pairs: 6; Success D0→D2: 50%→50%; Calls saved: -5.6%; Tokens saved: -28.2%; Time saved: +0.4%
- Frontier: Opus 4.7; Task: WebShop; Jev: 27B; Pairs: 10; Success D0→D2: 50%→60%; Calls saved: +7.7%; Tokens saved: +2.9%; Time saved: +6.1%
- Frontier: Sonnet 4.6; Task: FrozenLake; Jev: 27B; Pairs: 10; Success D0→D2: 100%→100%; Calls saved: +36.6%; Tokens saved: +34.4%; Time saved: -14.1%
- Frontier: Sonnet 4.6; Task: Sokoban; Jev: 27B; Pairs: 10; Success D0→D2: 90%→90%; Calls saved: -10.1%; Tokens saved: -14.8%; Time saved: +3.8%
- Frontier: GPT-5.6-sol; Task: FrozenLake; Jev: 27B; Pairs: 10; Success D0→D2: 100%→100%; Calls saved: +64.4%; Tokens saved: +63.1%; Time saved: +37.6%
- Frontier: GPT-5.6-sol; Task: Sokoban; Jev: 27B; Pairs: 10; Success D0→D2: 100%→100%; Calls saved: -26.2%; Tokens saved: -122.0%; Time saved: -7.5%

Only GPT-5.6-sol/FrozenLake is a clear multi-metric win: paired bootstrap intervals for call, token, and latency differences exclude zero. Its mean optional-minus-baseline differences (95% paired bootstrap intervals) are -2.9 calls [-4.7, -1.3], -1,929.6 tokens [-3,854.3, -427.7], and -5.55 seconds [-10.69, -0.30]. WebShop's point estimates are favorable, but its paired intervals for calls, tokens, and latency cross zero. Opus/FrozenLake saves model work but loses one success in each cell and is slower; it is not a win.

G. Open-terminal exploration

Terminal-Bench 2 uses Opus 4.7 as the frontier and the 27B Jev checkpoint. The v3 sample contains one stochastic D0/D3 pair for each of six predeclared tasks. It is protocol-development evidence, not a statistically powered benchmark.

- Stratum/task: easy / regex-log; Reward D0→D3: 0→1; Calls: 30→18; Tokens: 186,712→196,687; Time: 287.0s→178.8s; D3 command replacement: 5/10
- Stratum/task: easy / sqlite-db-truncate; Reward D0→D3: 1→1; Calls: 17→15; Tokens: 202,102→222,728; Time: 146.2s→163.8s; D3 command replacement: 9/13
- Stratum/task: medium / configure-git-webserver; Reward D0→D3: 0→0; Calls: 30→30; Tokens: 267,467→308,860; Time: 249.5s→326.0s; D3 command replacement: 23/31
- Stratum/task: medium / llm-inference-batching-scheduler; Reward D0→D3: 0→0; Calls: 30→30; Tokens: 572,053→600,218; Time: 210.6s→302.6s; D3 command replacement: 19/24
- Stratum/task: hard / query-optimize; Reward D0→D3: 0→1; Calls: 30→29; Tokens: 366,138→348,370; Time: 831.1s→937.1s; D3 command replacement: 27/29
- Stratum/task: hard / path-tracing; Reward D0→D3: 0→0; Calls: 30→30; Tokens: 574,798→361,174; Time: 558.0s→339.3s; D3 command replacement: 17/26

Aggregated descriptively, success is 1/6→3/6, with 9.0% calls, 6.0% tokens, 6.9% estimated Bed-rock cost, and 1.5% wall time saved. This aggregate must not be read causally: it combines heterogeneous tasks and only one attempt per cell. The full negative rows remain important. Easy sqlite-db-truncate is slower and more token-intensive in v3; both medium tasks consume more resources and still fail; hard path-tracing becomes cheaper but still does not complete.

The later v4 SQLite mechanism sweep holds verifier reward at 1 in all three single attempts:

- Target delegation: 0%; Actual eligible coverage: 0/2; Calls: 15; Tokens: 202,050; Estimated frontier cost: \$1.292; Time: 187.9s
- Target delegation: 50%; Actual eligible coverage: 1/4; Calls: 11; Tokens: 161,261; Estimated frontier cost: \$1.009; Time: 155.1s

- Target delegation: 100%; Actual eligible coverage: 2/3; Calls: 8; Tokens: 121,293; Estimated frontier cost: \$0.764; Time: 144.7s

Relative to this sampled 0% run, the 100% target reduces calls 46.7%, tokens 40.0%, estimated frontier cost 40.9%, and time 23.0%. The curve is a demo, not a confidence interval. A later strict-protocol (1.2.0) smoke also passes: reward 1, 10 frontier calls, 138,725 tokens, \$0.873 estimated frontier cost, and 91.8 seconds. Jev executes four of five eligible multi-option decisions and seven of ten commands; one low-confidence decision falls back to the frontier.

H. Decision demonstrations

H.1 SQLite recovery: a meaningful three-way decision

In the v4 candidate-logging sweep, the frontier LLM receives a truncated SQLite file and proposes three materially different diagnostic actions:

```
[0] od head
    od -A x -t x1z -v /app/trunc.db | head -80
    expected: raw hexadecimal page structure

[1] sqlite schema
    sqlite3 /app/trunc.db '.schema';
    sqlite3 /app/trunc.db 'SELECT * FROM sqlite_master;';
    expected: logical schema if the header is usable

[2] check tools
    which sqlite3 python3
    expected: available recovery tools
```

The LLM's safe fallback is option 0. Jev independently selects option 0 at 0.81 confidence. The result exposes a 4,096-byte page beginning with byte 0d, ten cells, content start 0x0f49, and ten cell-pointer offsets. The LLM then recognizes a SQLite leaf-table b-tree page, parses cell pointers, varints, and serial types, recovers testword00 through testword09, corrects an output construction error, and writes /app/recover.json. Harbor's independent verifier returns reward 1.

This division of labor is substantive but bounded: Jev chooses among raw-byte, logical-schema, and tool-availability paths; the LLM performs the recovery algorithm and completion verification. A separate targeted Jev return below 0.55 is rejected, and the harness executes the LLM fallback instead.

H.2 WebArena popup-aware navigation: useful but only a smoke

For WebArena task 264 ("browse products in Cabinets, Racks & Shelves"), both frontier-only and optional-Jev runs pass. The optional trace executes hover [887], hover [2641], and click [2661] under Jev control, then the LLM verifies the category and stops. In this one task, browser actions fall 7->4, frontier calls 8->2, tokens 76,674->13,407, and time 96.6->80.4 seconds.

This is a mechanism demo, not aggregate evidence. The frozen six-task WebArena row remains 50%->50% success with 5.6% more calls and 28.2% more tokens. The task 264 trace therefore cannot be used to hide the negative aggregate.

H.3 Query optimization: selection supports, but does not replace, expertise

In the v3 query-optimize run, Jev controls 27 of 29 commands covering schema inspection, data-size checks, duplicate/orphan/null diagnostics, timing, query execution, and output comparison. The LLM authors the decisive CTE/window- function rewrite. The new query reproduces the output and runs in about 0.56 seconds versus about 214 seconds for the original; the independent task reward changes 0->1. Yet agent wall time rises from 831 to 937 seconds. This is an exploratory quality-recovery example, not an end-to-end latency win, and v3 did not persist every unselected candidate text.

H.4 WebShop: a confident local mistake and frontier recovery

WebShop seed 3100 requests a slim-fit short-sleeve men's henley with exact color 155- blue, size x-large, and price below \$40. On an initially plausible product, Jev selects click[light blue] at 0.99

confidence. The environment reports the action as ineffective, and the selected color does not satisfy the exact global constraint. The frontier LLM consumes that observation, returns to search, opens a different product, selects exact 155- blue and x-large, and completes the purchase. The independent verifier returns reward 1.

This trace demonstrates a meaningful action choice and the need for retained frontier ownership. Confidence is not correctness; a local decision model can prefer a plausible near match while missing a task-level constraint. Immediate re-observation lets the LLM recover. The trace stores the executed Jev action but not its complete unselected menu, so it is a recovery demo rather than a fully replayable candidate-menu comparison. The ten-pair WebShop aggregate remains a positive but inconclusive signal because paired intervals cross zero.

I. Failure analysis and protocol evolution

The early results explain why the strict contract matters:

1. Over-delegation: v1 allowed too many commands after one LLM decision.
State changes invalidated later choices and inflated calls and tokens.
2. No amortization: calling the LLM before every basic action and then adding Jev introduces cost without replacing frontier work.
3. Artificial menus: requiring multiple choices for exact edits produced fake alternatives and schema-repair calls. Non-routine exact edits now allow one candidate; routine decisions require at least two.
4. Misclassification: the v4 regex-log sweep labeled all basic actions as non-routine singletons, yielding zero realized Jev coverage. Different configured rates then measured stochastic trajectory variation, not Jev.
5. Capability deficit: high local command replacement cannot rescue a frontier that lacks the global configuration or coding plan.
6. Cheap-task overhead: for short obvious actions, candidate generation, Jev inference, and context transfer cost more than direct execution.
7. Dynamic-state drift: in browsers or persistent terminals, a candidate can become stale after a popup, navigation, or side effect.

The final protocol consequently enforces bounded candidate menus, a short composability horizon, a 0.55 confidence fallback, complete candidate logging, independent verification, and separate configured versus realized coverage. Routine/non-routine semantics remain a frontier-model declaration; the local validator is a narrow policy gate, not a general shell-security boundary.

J. Claims, limitations, and next experiments

Supported claims:

- A Jev decision layer can preserve quality while reducing frontier work on some bounded, repetitive decisions.
- The repeated GPT-5.6-sol/FrozenLake cell demonstrates a controlled multi-metric efficiency win.
- Real agent environments exhibit both positive mechanisms and clear negative results; delegation rate alone does not predict success.

Not yet supported:

- universal agent improvement;
- a causal population-level Terminal-Bench success gain;

- a general WebArena or WebShop win;
- an efficiency claim that includes monetized Jev GPU cost;
- D4 long-horizon autonomous subgoal execution.

The next compact experiment should freeze the six Terminal-Bench tasks and run paired repetitions of D0, D1, and D3 under the final strict protocol. D1 can estimate agreement and counterfactual regret without execution risk. D4 should be attempted only on one easy and one medium task after D3 is stable. Report task-level distributions and paired intervals rather than only aggregate means, and add Jev service utilization/cost, candidate semantic-distance labels, and fallback confidence values. Do not expand the current WebArena sample after observing results; any expansion must be a separately versioned, predeclared study.

K. Reproducibility pointers

- Formal 96-pair matrix: results/agent-harness-v1/formal-matrix.md and .json
- Terminal-Bench v3 snapshot: results/agent-harness-v1/terminal-bench-sample-v3.md and .json
- Delegation-rate sweep: results/agent-harness-v1/terminal-bench-frontier-v4.md and .json
- Decision traces: Sections H.1 (SQLite), H.2 (WebArena), H.3 (query optimization), and H.4 (WebShop)
- Task/delegation protocol: docs/experiments/AGENT_HARNESS_FRONTIER_PROTOCOL.md

Raw sources, failed runs, development versions, exact model IDs, hashes, and selected-command traces are retained. Estimated dollar values use recorded frontier API pricing and exclude Jev GPU rental and energy.